Academy Hub — Modelo de datos completo

Documento técnico del modelo de dominio

Índice

1. Visión general de apps
2. App `accounts`
3. App `academies`
4. App `courses`
5. App `enrollments`
6. App `core`
7. Resumen de relaciones
8. Resumen de `on_delete`
9. Resumen de `related_name`
10. Resumen de restricciones únicas
11. Resumen de índices
12. Resumen de ordering
13. Diagrama entidad-relación
14. Orden recomendado de creación y migración
15. Notas de implementación

---

1. Visión general de apps

En esta fase del proyecto tenemos **cinco apps** implementadas:

App	Responsabilidad
`core`	Páginas generales, home, utilidades, contacto
`accounts`	Usuarios (academias y alumnos), autenticación, perfiles
`academies`	Academias y profesores
`courses`	Categorías, cursos, niveles, modalidades, convocatorias, imágenes
`enrollments`	Solicitudes, reservas, matrículas, lista de espera

Decisión importante: aunque en un primer momento podríamos pensar en `academies` y `courses` como una sola app, las separamos porque:

- `academies` gestiona la **identidad de la academia** y sus **profesores**.
- `courses` gestiona el **catálogo formativo** y sus **convocatorias**.
- `enrollments` gestiona **todo el ciclo del alumno** (solicitud → reserva → matrícula → baja).

Esta separación sigue el principio de **una app, una responsabilidad**.

---

2. App `accounts`

2.1. Modelo `User`

Extiende `AbstractUser` de Django.

Ubicación: `accounts/models.py`

```
from django.contrib.auth.models import AbstractUser
from django.db import models


class User(AbstractUser):
    """
    Usuario personalizado de Academy Hub.
    Puede ser academia o alumno.
    """

    email = models.EmailField(
        unique=True,
        verbose_name="Email",
    )

    is_academy = models.BooleanField(
        default=False,
        verbose_name="Es academia",
    )

    is_student = models.BooleanField(
        default=False,
        verbose_name="Es alumno",
    )

    avatar = models.ImageField(
        upload_to="avatars/",
        blank=True,
        null=True,
        verbose_name="Avatar",
    )

    city = models.CharField(
        max_length=100,
        blank=True,
        verbose_name="Ciudad",
    )

    bio = models.TextField(
        max_length=500,
```

```
        blank=True,
        verbose_name="Biografía",
    )

    phone = models.CharField(
        max_length=20,
        blank=True,
        verbose_name="Teléfono",
    )

    is_verified = models.BooleanField(
        default=False,
        verbose_name="Verificado",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de creación",
    )

    updated_at = models.DateTimeField(
        auto_now=True,
        verbose_name="Última actualización",
    )

    class Meta:
        verbose_name = "Usuario"
        verbose_name_plural = "Usuarios"
        ordering = ["-created_at"]

    def __str__(self):
        return self.username
```

Notas:

- `email` es `unique=True` a nivel de modelo.
- `is_academy` e `is_student` permiten diferenciar roles sin necesidad de grupos.
- `is_verified` se usará más adelante para academias verificadas.
- `created_at` / `updated_at` son trazabilidad básica.

Configuración obligatoria en `settings/base.py`:

```
AUTH_USER_MODEL = "accounts.User"
```

---

3. App `academies`

3.1. Modelo `Academy`

Ubicación: `academies/models.py`

Representa una academia real. Cada academia pertenece a un `User` con `is_academy=True`.

```
from django.conf import settings
from django.db import models
from django.utils.text import slugify


class Academy(models.Model):
    """
    Academia que publica cursos en Academy Hub.
    """

    owner = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="academies",
        verbose_name="Propietario",
    )

    name = models.CharField(
        max_length=150,
        unique=True,
        verbose_name="Nombre",
    )

    slug = models.SlugField(
        max_length=180,
        unique=True,
        blank=True,
        verbose_name="Slug",
    )

    description = models.TextField(
        blank=True,
        verbose_name="Descripción",
    )

    city = models.CharField(
        max_length=100,
        blank=True,
        verbose_name="Ciudad",
    )

    address = models.CharField(
        max_length=200,
        blank=True,
        verbose_name="Dirección",
    )

    website = models.URLField(
        blank=True,
        verbose_name="Sitio web",
    )
```

```
    phone = models.CharField(
        max_length=20,
        blank=True,
        verbose_name="Teléfono",
    )

    email = models.EmailField(
        blank=True,
        verbose_name="Email de contacto",
    )

    logo = models.ImageField(
        upload_to="academies/logos/",
        blank=True,
        null=True,
        verbose_name="Logo",
    )

    is_verified = models.BooleanField(
        default=False,
        verbose_name="Verificada",
    )

    is_active = models.BooleanField(
        default=True,
        verbose_name="Activa",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de creación",
    )

    updated_at = models.DateTimeField(
        auto_now=True,
        verbose_name="Última actualización",
    )

    class Meta:
        verbose_name = "Academia"
        verbose_name_plural = "Academias"
        ordering = ["-created_at"]
        indexes = [
            models.Index(fields=["city"]),
            models.Index(fields=["is_active"]),
            models.Index(fields=["is_verified"]),
        ]

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)
```

```
    def __str__(self):
        return self.name
```

Campos clave:

- `owner` : propietario (`User`).
- `name` y `slug` : únicos.
- `is_verified` : para mostrar el check de academia verificada.
- `is_active` : para ocultar academias sin borrarlas.

Índices:

- `city` , `is_active` , `is_verified` .

3.2. Modelo `Teacher`

Ubicación: `academies/models.py`

Un profesor pertenece siempre a una academia.

```
class Teacher(models.Model):
    """
    Profesor de una academia.
    """

    academy = models.ForeignKey(
        Academy,
        on_delete=models.CASCADE,
        related_name="teachers",
        verbose_name="Academia",
    )

    first_name = models.CharField(
        max_length=100,
        verbose_name="Nombre",
    )

    last_name = models.CharField(
        max_length=100,
        blank=True,
        verbose_name="Apellidos",
    )

    email = models.EmailField(
        blank=True,
        verbose_name="Email",
    )

    phone = models.CharField(
```

```
        max_length=20,
        blank=True,
        verbose_name="Teléfono",
    )

    specialty = models.CharField(
        max_length=150,
        blank=True,
        verbose_name="Especialidad",
    )

    bio = models.TextField(
        blank=True,
        verbose_name="Biografía",
    )

    photo = models.ImageField(
        upload_to="teachers/",
        blank=True,
        null=True,
        verbose_name="Foto",
    )

    is_active = models.BooleanField(
        default=True,
        verbose_name="Activo",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de creación",
    )

    updated_at = models.DateTimeField(
        auto_now=True,
        verbose_name="Última actualización",
    )

    class Meta:
        verbose_name = "Profesor"
        verbose_name_plural = "Profesores"
        ordering = ["last_name", "first_name"]
        indexes = [
            models.Index(fields=["academy"]),
            models.Index(fields=["is_active"]),
        ]

    def __str__(self):
        return f"{self.first_name} {self.last_name}".strip()
```

Notas:

- `academy` es `CASCADE`: si desaparece la academia, desaparecen sus profesores.

- Ordenación alfabética por apellido.

---

4. App `courses`

4.1. Modelo `Category`

Ubicación: `courses/models.py`

```
from django.db import models
from django.utils.text import slugify


class Category(models.Model):
    """
    Categoría de curso (Idiomas, Informática, Oposiciones...).
    """

    name = models.CharField(
        max_length=100,
        unique=True,
        verbose_name="Nombre",
    )

    slug = models.SlugField(
        max_length=120,
        unique=True,
        blank=True,
        verbose_name="Slug",
    )

    description = models.TextField(
        blank=True,
        verbose_name="Descripción",
    )

    icon = models.CharField(
        max_length=50,
        blank=True,
        verbose_name="Icono",
        help_text="Nombre del icono (ej: book, code, music)",
    )

    is_active = models.BooleanField(
        default=True,
        verbose_name="Activa",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de creación",
```

```
    )

    class Meta:
        verbose_name = "Categoría"
        verbose_name_plural = "Categorías"
        ordering = ["name"]

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)

    def __str__(self):
        return self.name
```

Notas:

- `name` y `slug` únicos.
- `icon` es un placeholder para el frontend.
- Ordenación alfabética.

4.2. Choices: `CourseLevel`, `CourseModality`, `CourseStatus`, `EnrollmentMode`

Ubicación: `courses/models.py`

```
class CourseLevel(models.TextChoices):
    BEGINNER = "beginner", "Básico"
    INTERMEDIATE = "intermediate", "Intermedio"
    ADVANCED = "advanced", "Avanzado"
    EXPERT = "expert", "Experto"


class CourseModality(models.TextChoices):
    PRESENCIAL = "presencial", "Presencial"
    ONLINE = "online", "Online"
    HYBRID = "hybrid", "Híbrida"


class CourseStatus(models.TextChoices):
    DRAFT = "draft", "Borrador"
    PUBLISHED = "published", "Publicado"
    ARCHIVED = "archived", "Archivado"


class EnrollmentMode(models.TextChoices):
    DIRECT = "direct", "Inscripción directa"
    APPROVAL = "approval", "Admisión previa"
```

Notas:

- `EnrollmentMode` se usará en `Course` para decidir si la inscripción es directa o requiere admisión.
- Todos son `TextChoices`, por lo que se almacenan como strings cortos.

4.3. Modelo `Course`

Ubicación: `courses/models.py`

```
from django.core.validators import MinValueValidator


class Course(models.Model):
    """
    Curso ofertado por una academia.
    """

    academy = models.ForeignKey(
        "academies.Academy",
        on_delete=models.CASCADE,
        related_name="courses",
        verbose_name="Academia",
    )

    category = models.ForeignKey(
        Category,
        on_delete=models.PROTECT,
        related_name="courses",
        verbose_name="Categoría",
    )

    teacher = models.ForeignKey(
        "academies.Teacher",
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="courses",
        verbose_name="Profesor",
    )

    title = models.CharField(
        max_length=180,
        verbose_name="Título",
    )

    slug = models.SlugField(
        max_length=200,
        blank=True,
        verbose_name="Slug",
    )

    description = models.TextField(
        verbose_name="Descripción",
```

```
    )

    objectives = models.TextField(
        blank=True,
        verbose_name="Objetivos",
    )

    competences = models.TextField(
        blank=True,
        verbose_name="Competencias",
        help_text="Competencias que desarrolla el curso",
    )

    requirements = models.TextField(
        blank=True,
        verbose_name="Requisitos previos",
    )

    level = models.CharField(
        max_length=20,
        choices=CourseLevel.choices,
        default=CourseLevel.BEGINNER,
        verbose_name="Nivel",
    )

    modality = models.CharField(
        max_length=20,
        choices=CourseModality.choices,
        default=CourseModality.PRESENCIAL,
        verbose_name="Modalidad",
    )

    status = models.CharField(
        max_length=20,
        choices=CourseStatus.choices,
        default=CourseStatus.DRAFT,
        verbose_name="Estado",
    )

    enrollment_mode = models.CharField(
        max_length=20,
        choices=EnrollmentMode.choices,
        default=EnrollmentMode.DIRECT,
        verbose_name="Modo de inscripción",
    )

    requires_level_test = models.BooleanField(
        default=False,
        verbose_name="Requiere prueba de nivel",
    )

    is_negotiable = models.BooleanField(
        default=False,
```

```
    verbose_name="Precio negociable",
)

duration_hours = models.PositiveIntegerField(
    default=0,
    verbose_name="Duración (horas)",
)

views_count = models.PositiveIntegerField(
    default=0,
    verbose_name="Visitas",
)

created_at = models.DateTimeField(
    auto_now_add=True,
    verbose_name="Fecha de creación",
)

updated_at = models.DateTimeField(
    auto_now=True,
    verbose_name="Última actualización",
)

class Meta:
    verbose_name = "Curso"
    verbose_name_plural = "Cursos"
    ordering = ["-created_at"]
    indexes = [
        models.Index(fields=["status"]),
        models.Index(fields=["created_at"]),
        models.Index(fields=["level"]),
        models.Index(fields=["modality"]),
        models.Index(fields=["category"]),
        models.Index(fields=["academy"]),
    ]

def save(self, *args, **kwargs):
    if not self.slug:
        self.slug = slugify(self.title)
    super().save(*args, **kwargs)

def __str__(self):
    return self.title

@property
def primary_image(self):
    primary = self.images.filter(is_primary=True).first()
    if primary:
        return primary
    return self.images.first()

@property
def is_published(self):
```

```
        return self.status == CourseStatus.PUBLISHED

    @property
    def has_convocatorias(self):
        return self.convocatorias.filter(is_active=True).exists()
```

Notas importantes:

- `slug` **no es único**: dos cursos con el mismo título en academias distintas pueden coexistir. La unicidad pública se resolverá con `id` + `slug` en la URL.
- `category` usa `PROTECT`: no se puede borrar una categoría con cursos.
- `teacher` usa `SET_NULL`: si se borra un profesor, el curso sobrevive.
- `academy` usa `CASCADE`.

Índices:

- `status`, `created_at`, `level`, `modality`, `category`, `academy`.

Propiedades:

- `primary_image`, `is_published`, `has_convocatorias`.

4.4. Modelo `Convocatoria`

Ubicación: `courses/models.py`

Una convocatoria es una **edición concreta** de un curso.

```
class Convocatoria(models.Model):
    """
    Edición concreta de un curso con fechas, plazas y precio.
    """

    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="convocatorias",
        verbose_name="Curso",
    )

    name = models.CharField(
        max_length=150,
        blank=True,
        verbose_name="Nombre",
        help_text="Ej: Verano 2026, Grupo A, Mañanas",
    )

    start_date = models.DateField(
        verbose_name="Fecha de inicio",
    )
```

```
    end_date = models.DateField(
        verbose_name="Fecha de fin",
    )

    enrollment_deadline = models.DateField(
        null=True,
        blank=True,
        verbose_name="Fecha límite de inscripción",
    )

    schedule = models.CharField(
        max_length=200,
        blank=True,
        verbose_name="Horario",
        help_text="Ej: L-X-V 18:00-20:00",
    )

    modality = models.CharField(
        max_length=20,
        choices=CourseModality.choices,
        default=CourseModality.PRESENCIAL,
        verbose_name="Modalidad",
    )

    location = models.CharField(
        max_length=200,
        blank=True,
        verbose_name="Ubicación",
    )

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0)],
        verbose_name="Precio",
    )

    deposit = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        default=0,
        validators=[MinValueValidator(0)],
        verbose_name="Señal / Reserva",
        help_text="Cantidad a pagar para reservar plaza",
    )

    total_places = models.PositiveIntegerField(
        default=0,
        verbose_name="Plazas totales",
    )

    reserved_places = models.PositiveIntegerField(
```

```
    default=0,
    verbose_name="Plazas reservadas",
)

enrolled_places = models.PositiveIntegerField(
    default=0,
    verbose_name="Plazas matriculadas",
)

is_active = models.BooleanField(
    default=True,
    verbose_name="Activa",
)

created_at = models.DateTimeField(
    auto_now_add=True,
    verbose_name="Fecha de creación",
)

updated_at = models.DateTimeField(
    auto_now=True,
    verbose_name="Última actualización",
)

class Meta:
    verbose_name = "Convocatoria"
    verbose_name_plural = "Convocatorias"
    ordering = ["start_date"]
    indexes = [
        models.Index(fields=["start_date"]),
        models.Index(fields=["is_active"]),
        models.Index(fields=["course"]),
    ]

def __str__(self):
    label = self.name or f"{self.start_date}"
    return f"{self.course.title} — {label}"

@property
def available_places(self):
    used = self.reserved_places + self.enrolled_places
    return max(self.total_places - used, 0)

@property
def is_full(self):
    return self.available_places == 0

@property
def is_open(self):
    from django.utils import timezone
    if not self.is_active:
        return False
    if self.is_full:
```

```
            return False
        if self.enrollment_deadline:
            if timezone.now().date() > self.enrollment_deadline:
                return False
        return True
```

Notas:

- Se separan `reserved_places` y `enrolled_places` para distinguir entre reservas temporales y matrículas confirmadas.
- `available_places` se calcula.
- `is_open` comprueba activa + no llena + dentro de plazo.

4.5. Modelo `CourseImage`

Ubicación: `courses/models.py`

```
class CourseImage(models.Model):
    """
    Imagen asociada a un curso.
    """

    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="images",
        verbose_name="Curso",
    )

    image = models.ImageField(
        upload_to="courses/",
        verbose_name="Imagen",
    )

    alt_text = models.CharField(
        max_length=200,
        blank=True,
        verbose_name="Texto alternativo",
    )

    is_primary = models.BooleanField(
        default=False,
        verbose_name="Imagen principal",
    )

    position = models.PositiveSmallIntegerField(
        default=0,
        verbose_name="Posición",
    )
```

```
    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de creación",
    )

    class Meta:
        verbose_name = "Imagen de curso"
        verbose_name_plural = "Imágenes de curso"
        ordering = ["position", "created_at"]
        indexes = [
            models.Index(fields=["course", "is_primary"]),
        ]

    def __str__(self):
        return f"Imagen de {self.course.title}"
```

---

5. App `enrollments`

5.1. Choices de estado

Ubicación: `enrollments/models.py`

```
from django.db import models


class EnrollmentStatus(models.TextChoices):
    PENDING = "pending", "Solicitud pendiente"
    APPROVED = "approved", "Admitido"
    REJECTED = "rejected", "Rechazado"
    RESERVED = "reserved", "Reserva activa"
    RESERVATION_EXPIRED = "reservation_expired", "Reserva caducada"
    ENROLLED = "enrolled", "Matriculado"
    WAITLIST = "waitlist", "Lista de espera"
    CANCELLED = "cancelled", "Cancelado"
    COMPLETED = "completed", "Completado"


class PaymentStatus(models.TextChoices):
    PENDING = "pending", "Pendiente"
    PARTIAL = "partial", "Pago parcial"
    PAID = "paid", "Pagado"
    REFUNDED = "refunded", "Reembolsado"
    FAILED = "failed", "Fallido"
```

5.2. Modelo `Enrollment`

Ubicación: `enrollments/models.py`

Representa **el ciclo completo del alumno** en una convocatoria concreta.

```
from django.conf import settings
from django.core.validators import MinValueValidator
from django.db import models
from django.utils import timezone

from courses.models import Convocatoria


class Enrollment(models.Model):
    """
    Solicitud / reserva / matrícula de un alumno en una convocatoria.
    """

    student = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="enrollments",
        verbose_name="Alumno",
    )

    convocatoria = models.ForeignKey(
        Convocatoria,
        on_delete=models.CASCADE,
        related_name="enrollments",
        verbose_name="Convocatoria",
    )

    status = models.CharField(
        max_length=30,
        choices=EnrollmentStatus.choices,
        default=EnrollmentStatus.PENDING,
        verbose_name="Estado",
    )

    payment_status = models.CharField(
        max_length=20,
        choices=PaymentStatus.choices,
        default=PaymentStatus.PENDING,
        verbose_name="Estado de pago",
    )

    amount_paid = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        default=0,
        validators=[MinValueValidator(0)],
        verbose_name="Importe pagado",
    )

    discount_amount = models.DecimalField(
        max_digits=10,
```

```
    decimal_places=2,
    default=0,
    validators=[MinValueValidator(0)],
    verbose_name="Descuento aplicado",
)

reservation_expires_at = models.DateTimeField(
    null=True,
    blank=True,
    verbose_name="La reserva caduca el",
)

enrolled_at = models.DateTimeField(
    null=True,
    blank=True,
    verbose_name="Fecha de matrícula",
)

completed_at = models.DateTimeField(
    null=True,
    blank=True,
    verbose_name="Fecha de finalización",
)

notes = models.TextField(
    blank=True,
    verbose_name="Notas internas",
)

created_at = models.DateTimeField(
    auto_now_add=True,
    verbose_name="Fecha de creación",
)

updated_at = models.DateTimeField(
    auto_now=True,
    verbose_name="Última actualización",
)

class Meta:
    verbose_name = "Inscripción"
    verbose_name_plural = "Inscripciones"
    ordering = ["-created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["student", "convocatoria"],
            name="unique_student_convocatoria",
        ),
    ]
    indexes = [
        models.Index(fields=["status"]),
        models.Index(fields=["payment_status"]),
        models.Index(fields=["student", "status"]),
```

```
            models.Index(fields=["convocatoria", "status"]),
        ]

    def __str__(self):
        return f"{self.student.username} → {self.convocatoria}"

    @property
    def is_active(self):
        return self.status in [
            EnrollmentStatus.PENDING,
            EnrollmentStatus.APPROVED,
            EnrollmentStatus.RESERVED,
            EnrollmentStatus.ENROLLED,
            EnrollmentStatus.WAITLIST,
        ]

    @property
    def is_reservation_expired(self):
        if not self.reservation_expires_at:
            return False
        return timezone.now() > self.reservation_expires_at

    @property
    def total_price(self):
        return self.convocatoria.price - self.discount_amount

    @property
    def pending_amount(self):
        return max(self.total_price - self.amount_paid, 0)
```

Notas clave:

- `UniqueConstraint` impide que un alumno se inscriba dos veces en la misma convocatoria.
- `reservation_expires_at` gestiona la caducidad de la reserva.
- `EnrollmentStatus` cubre todo el ciclo.
- `payment_status` cubre la parte económica.

5.3. Modelo `WaitlistEntry`

Ubicación: `enrollments/models.py`

Lista de espera para convocatorias llenas.

```
class WaitlistEntry(models.Model):
    """
    Entrada en lista de espera para una convocatoria llena.
    """

    student = models.ForeignKey(
        settings.AUTH_USER_MODEL,
```

```
    on_delete=models.CASCADE,
    related_name="waitlist_entries",
    verbose_name="Alumno",
)

convocatoria = models.ForeignKey(
    Convocatoria,
    on_delete=models.CASCADE,
    related_name="waitlist_entries",
    verbose_name="Convocatoria",
)

position = models.PositiveIntegerField(
    default=0,
    verbose_name="Posición",
)

is_notified = models.BooleanField(
    default=False,
    verbose_name="Notificado",
)

notified_at = models.DateTimeField(
    null=True,
    blank=True,
    verbose_name="Fecha de notificación",
)

converted_to_enrollment = models.ForeignKey(
    Enrollment,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="waitlist_source",
    verbose_name="Inscripción generada",
)

created_at = models.DateTimeField(
    auto_now_add=True,
    verbose_name="Fecha de alta en lista",
)

class Meta:
    verbose_name = "Lista de espera"
    verbose_name_plural = "Listas de espera"
    ordering = ["position", "created_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["student", "convocatoria"],
            name="unique_waitlist_student_convocatoria",
        ),
    ]
    indexes = [
```

```
            models.Index(fields=["convocatoria", "position"]),
        ]

    def __str__(self):
        return f"#{self.position} {self.student.username} → {self.convocatoria}"
```

5.4. Modelo `EnrollmentAnswer`

Ubicación: `enrollments/models.py`

Respuestas del alumno a las preguntas del formulario configurable.

```
class EnrollmentAnswer(models.Model):
    """
    Respuesta del alumno a una pregunta del formulario de inscripción.
    """

    enrollment = models.ForeignKey(
        Enrollment,
        on_delete=models.CASCADE,
        related_name="answers",
        verbose_name="Inscripción",
    )

    question_key = models.CharField(
        max_length=100,
        verbose_name="Clave de la pregunta",
    )

    question_text = models.CharField(
        max_length=300,
        verbose_name="Texto de la pregunta",
    )

    answer_text = models.TextField(
        blank=True,
        verbose_name="Respuesta",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de respuesta",
    )

    class Meta:
        verbose_name = "Respuesta de inscripción"
        verbose_name_plural = "Respuestas de inscripción"
        ordering = ["created_at"]
        indexes = [
            models.Index(fields=["enrollment"]),
        ]
```

```
    def __str__(self):
        return f"{self.question_key}: {self.answer_text[:40]}"
```

Nota: este modelo guarda las respuestas de forma flexible (clave + texto) para permitir **formularios configurables** por convocatoria sin necesidad de definir preguntas rígidas en base de datos en esta fase.

5.5. Modelo `EnrollmentDocument`

Ubicación: `enrollments/models.py`

Documentos adjuntos subidos por el alumno durante la inscripción.

```
class EnrollmentDocument(models.Model):
    """
    Documento adjunto a una inscripción.
    """

    enrollment = models.ForeignKey(
        Enrollment,
        on_delete=models.CASCADE,
        related_name="documents",
        verbose_name="Inscripción",
    )

    name = models.CharField(
        max_length=150,
        verbose_name="Nombre del documento",
    )

    file = models.FileField(
        upload_to="enrollments/documents/",
        verbose_name="Archivo",
    )

    is_verified = models.BooleanField(
        default=False,
        verbose_name="Verificado",
    )

    uploaded_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de subida",
    )

    class Meta:
        verbose_name = "Documento de inscripción"
        verbose_name_plural = "Documentos de inscripción"
        ordering = ["uploaded_at"]
```

```
    def __str__(self):
        return f"{self.name} ({self.enrollment})"
```

---

6. App `core`

6.1. Modelo `ContactMessage`

Ubicación: `core/models.py`

Aunque `core` es principalmente para páginas generales, tiene sentido almacenar mensajes de contacto o sugerencias.

```
from django.db import models


class ContactMessage(models.Model):
    """
    Mensaje enviado desde el formulario de contacto.
    """

    name = models.CharField(
        max_length=150,
        verbose_name="Nombre",
    )

    email = models.EmailField(
        verbose_name="Email",
    )

    subject = models.CharField(
        max_length=200,
        verbose_name="Asunto",
    )

    message = models.TextField(
        verbose_name="Mensaje",
    )

    is_read = models.BooleanField(
        default=False,
        verbose_name="Leído",
    )

    created_at = models.DateTimeField(
        auto_now_add=True,
        verbose_name="Fecha de envío",
    )

    class Meta:
        verbose_name = "Mensaje de contacto"
```

```
        verbose_name_plural = "Mensajes de contacto"
        ordering = ["-created_at"]

    def __str__(self):
        return f"{self.subject} — {self.name}"
```

---

7. Resumen de relaciones

Modelo	Relación	Destino	Tipo	related_name
`User`	—	—	—	—
`Academy`	owner	`User`	FK	`academies`
`Teacher`	academy	`Academy`	FK	`teachers`
`Category`	—	—	—	—
`Course`	academy	`Academy`	FK	`courses`
`Course`	category	`Category`	FK	`courses`
`Course`	teacher	`Teacher`	FK	`courses`
`Convocatoria`	course	`Course`	FK	`convocatorias`
`CourseImage`	course	`Course`	FK	`images`
`Enrollment`	student	`User`	FK	`enrollments`
`Enrollment`	convocatoria	`Convocatoria`	FK	`enrollments`
`WaitlistEntry`	student	`User`	FK	`waitlist_entries`
`WaitlistEntry`	convocatoria	`Convocatoria`	FK	`waitlist_entries`
`WaitlistEntry`	converted_to_enrollment	`Enrollment`	FK	`waitlist_source`
`EnrollmentAnswer`	enrollment	`Enrollment`	FK	`answers`
`EnrollmentDocument`	enrollment	`Enrollment`	FK	`documents`
`ContactMessage`	—	—	—	—

---

8. Resumen de `on_delete`

Modelo	Campo	on_delete	Motivo
`Academy`	owner	`CASCADE`	Si se borra el usuario, sus academias también
`Teacher`	academy	`CASCADE`	Los profesores dependen de la academia
`Course`	academy	`CASCADE`	Los cursos dependen de la academia
`Course`	category	`PROTECT`	No se puede borrar una categoría con cursos
`Course`	teacher	`SET_NULL`	Si se borra el profesor, el curso sigue
`Convocatoria`	course	`CASCADE`	Las convocatorias dependen del curso
`CourseImage`	course	`CASCADE`	Las imágenes dependen del curso
`Enrollment`	student	`CASCADE`	Si se borra el alumno, sus inscripciones también
`Enrollment`	convocatoria	`CASCADE`	Si se borra la convocatoria, sus inscripciones también
`WaitlistEntry`	student	`CASCADE`	Si se borra el alumno, sus entradas de lista también
`WaitlistEntry`	convocatoria	`CASCADE`	Si se borra la convocatoria, sus listas también
`WaitlistEntry`	converted_to_enrollment	`SET_NULL`	Si se borra la inscripción, la entrada sobrevive

Modelo	Campo	on_delete	Motivo
EnrollmentAnswer	enrollment	CASCADE	Las respuestas dependen de la inscripción
EnrollmentDocument	enrollment	CASCADE	Los documentos dependen de la inscripción

9. Resumen de `related_name`

Modelo origen	related_name	Acceso inverso
Academy.owner	academies	user.academies.all()
Teacher.academy	teachers	academy.teachers.all()
Course.academy	courses	academy.courses.all()
Course.category	courses	category.courses.all()
Course.teacher	courses	teacher.courses.all()
Convocatoria.course	convocatorias	course.convocatorias.all()
CourseImage.course	images	course.images.all()
Enrollment.student	enrollments	user.enrollments.all()
Enrollment.convocatoria	enrollments	convocatoria.enrollments.all()
WaitlistEntry.student	waitlist_entries	user.waitlist_entries.all()
WaitlistEntry.convocatoria	waitlist_entries	convocatoria.waitlist_entries.all()
EnrollmentAnswer.enrollment	answers	enrollment.answers.all()
EnrollmentDocument.enrollment	documents	enrollment.documents.all()

10. Resumen de restricciones únicas

Modelo	Restricción	Tipo
User	email	unique=True
Academy	name	unique=True
Academy	slug	unique=True
Category	name	unique=True
Category	slug	unique=True
Enrollment	(student, convocatoria)	UniqueConstraint
WaitlistEntry	(student, convocatoria)	UniqueConstraint

11. Resumen de índices

Modelo	Campos indexados
Academy	city , is_active , is_verified
Teacher	academy , is_active
Course	status , created_at , level , modality , category , academy
Convocatoria	start_date , is_active , course
CourseImage	(course, is_primary)
Enrollment	status , payment_status , (student, status) , (convocatoria, status)
WaitlistEntry	(convocatoria, position)
EnrollmentAnswer	enrollment

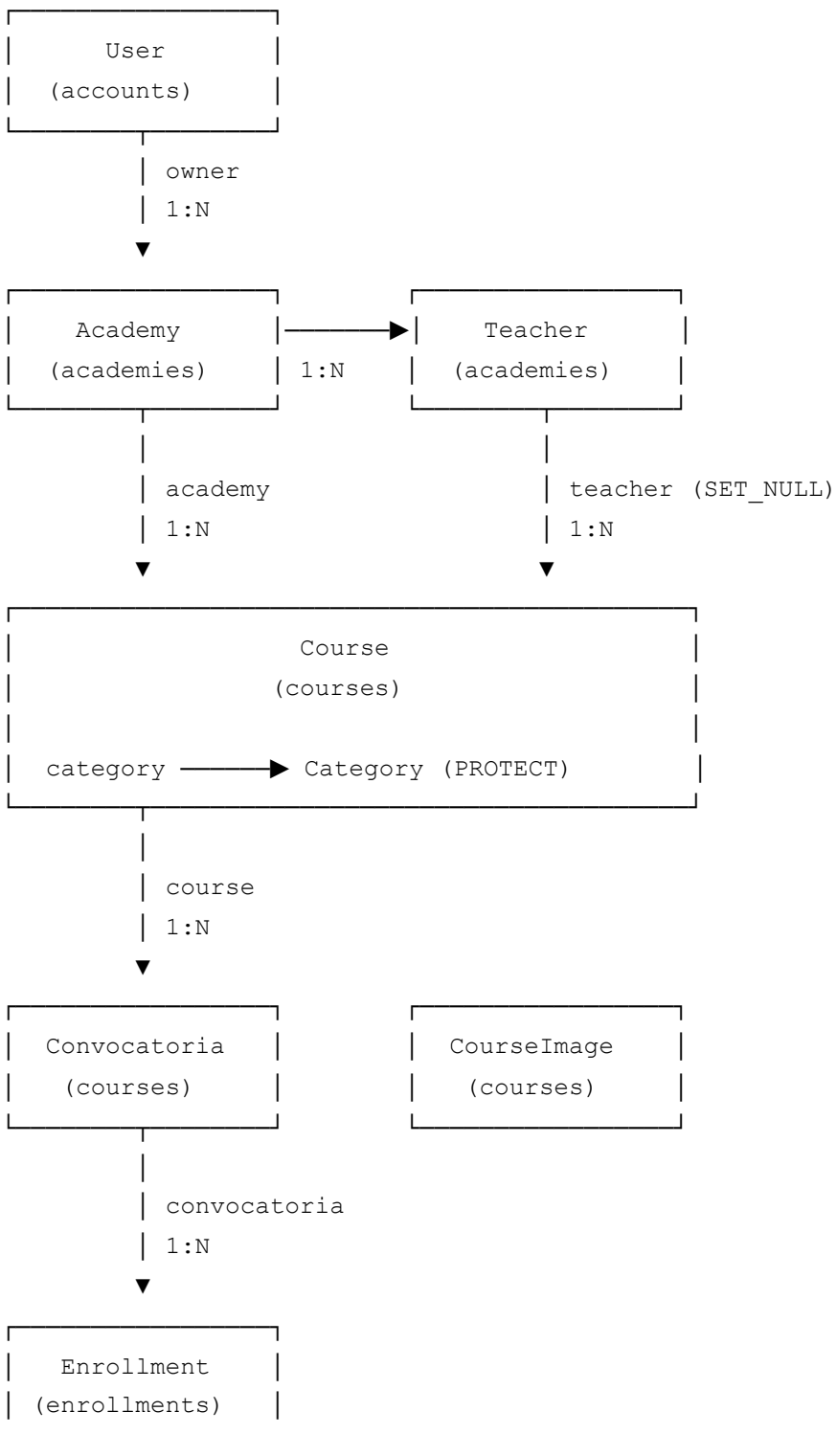
12. Resumen de ordering

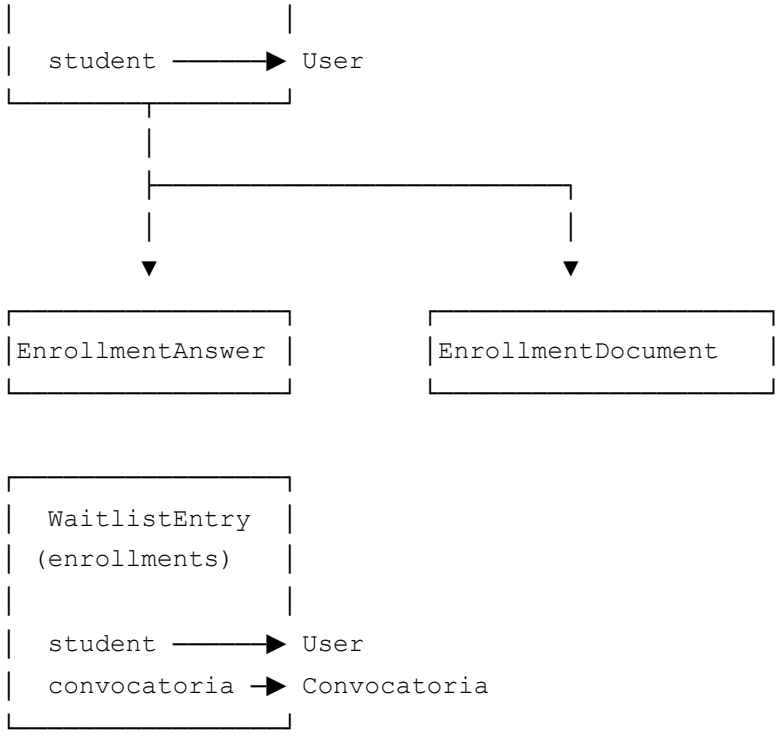
Modelo	ordering

Modelo	ordering
User	["-created_at"]
Academy	["-created_at"]
Teacher	["last_name", "first_name"]
Category	["name"]
Course	["-created_at"]
Convocatoria	["start_date"]
CourseImage	["position", "created_at"]
Enrollment	["-created_at"]
WaitlistEntry	["position", "created_at"]
EnrollmentAnswer	["created_at"]
EnrollmentDocument	["uploaded_at"]
ContactMessage	["-created_at"]

---

13. Diagrama entidad-relación





---

14. Orden recomendado de creación y migración

Las apps deben migrarse en orden porque hay dependencias entre ellas.

Orden:

1. `accounts` — debe ser la primera porque define `AUTH_USER_MODEL`.
2. `core` — no depende de nadie (solo `ContactMessage`).
3. `academies` — depende de `accounts`.
4. `courses` — depende de `academies`.
5. `enrollments` — depende de `accounts` y `courses`.

Comandos:

```
# 1. Resetear base de datos en desarrollo (solo si hace falta)
rm db.sqlite3

# 2. Borrar migraciones antiguas de apps propias (menos __init__.py)
#    accounts, academies, courses, enrollments, core

# 3. Crear migraciones en orden
python manage.py makemigrations accounts
python manage.py makemigrations core
python manage.py makemigrations academies
python manage.py makemigrations courses
python manage.py makemigrations enrollments

# 4. Aplicar
python manage.py migrate

# 5. Comprobar
```

```
python manage.py check

# 6. Crear superusuario
python manage.py createsuperuser
```

Nota: si Django se queja de dependencias circulares, se pueden crear todas las migraciones a la vez con:

```
python manage.py makemigrations
```

Django resolverá las dependencias internamente.

---

15. Notas de implementación

15.1. `AUTH_USER_MODEL`

Debe estar configurado **antes** de la primera migración:

```
# config/settings/base.py
AUTH_USER_MODEL = "accounts.User"
```

15.2. `INSTALLED_APPS`

```
LOCAL_APPS = [
    "core",
    "accounts",
    "academies",
    "courses",
    "enrollments",
]

INSTALLED_APPS = (
    DJANGO_APPS
    + THIRD_PARTY_APPS
    + LOCAL_APPS
)
```

15.3. Dependencias externas

- `Pillow` para `ImageField` y `FileField`:

```
pip install pillow
pip freeze > requirements.txt
```

15.4. Decisiones de diseño a tener en cuenta

- **`Course.slug` no es único**: la unicidad real es `(academy, slug)` a nivel de URL pública, pero se resuelve en la vista con `id` + `slug`.
- **`Convocatoria`** separa `reserved_places` y `enrolled_places` porque una reserva no es una matrícula.
- **`Enrollment`** tiene un `UniqueConstraint` para impedir duplicados por alumno y convocatoria.
- **`WaitlistEntry`** tiene su propio `UniqueConstraint` por alumno y convocatoria.
- **`EnrollmentAnswer`** guarda respuestas de forma flexible (clave + texto) para permitir formularios configurables sin necesidad de un modelo de "pregunta" rígido en esta fase.
- **`EnrollmentDocument`** guarda documentos adjuntos con estado de verificación.
- **`ContactMessage`** en `core` sirve para el formulario de contacto general.

15.5. Qué NO se ha incluido todavía

Para mantener el alcance razonable, **no** se han incluido aún:

- `Payment` (transacciones individuales)
- `Discount` / `Scholarship` (becas y descuentos como entidades)
- `Evaluation` (evaluación inicial/final)
- `TalentPassport` (Pasaporte de Talento)
- `LevelTest` (prueba de nivel)
- `Notification`
- `Review` (valoraciones)
- `CourseQuestion` / `CourseFormConfig` (formularios configurables con preguntas en base de datos)

Estos se añadirán en sesiones posteriores cuando se implementen sus funcionalidades correspondientes.

15.6. Próximos pasos

Con estos modelos ya se puede:

- Registrar academias y alumnos.
- Crear academias y profesores desde el admin.
- Crear categorías y cursos.
- Crear convocatorias con fechas, precios y plazas.
- Empezar a modelar inscripciones, reservas y listas de espera.
- Preparar el CRUD de cursos (Sesión 5).

Resumen final

App	Modelos
`accounts`	`User`
`academies`	`Academy`, `Teacher`
`courses`	`Category`, `Course`, `Convocatoria`, `CourseImage`

App	Modelos
`enrollments`	`Enrollment`, `WaitlistEntry`, `EnrollmentAnswer`, `EnrollmentDocument`
`core`	`ContactMessage`

Total: 12 modelos que cubren:

- Usuarios (academias y alumnos)
- Academias y profesores
- Catálogo formativo (categorías, cursos, convocatorias, imágenes)
- Ciclo del alumno (solicitud, reserva, matrícula, lista de espera, respuestas, documentos)
- Contacto general

Esta es la base sólida sobre la que se construirán las siguientes sesiones: CRUD de cursos, inscripciones, pagos, evaluaciones y Pasaporte de Talento.